

GOLDEN HOUR

Documento de Especificación de Requisitos



1. ¿De qué va este proyecto?

Seguramente hayas oído hablar de la "Hora de Oro" en series médicas o en la tele. El concepto es real y se usa desde los años 80 en medicina de urgencias: si una persona sufre un accidente grave, sus posibilidades de sobrevivir se multiplican si recibe atención médica especializada en los primeros 60 minutos.

El problema es que en España, y especialmente en regiones como Extremadura, hay zonas donde esto es difícil de garantizar. Extremadura tiene una extensión enorme (más de 41.000 km²) pero pocos hospitales de referencia, y hay puntos de sus carreteras desde los que llegas al hospital más cercano en más de una hora.



La idea del proyecto en una frase:

Golden Hour es una aplicación web que te muestra el mapa de tu ruta coloreado según si estás a menos de 30 minutos (verde), entre 30 y 60 minutos (naranja) o a más de 60 minutos (rojo) del hospital más cercano.

Piénsalo como un GPS, pero en lugar de avisarte de radares, te avisa de zonas donde estarías muy lejos de un hospital en caso de emergencia. La aplicación sugiere además paradas o rutas alternativas si detecta tramos críticos.

El proyecto no busca sustituir al 112 ni a los servicios de emergencias. Su objetivo es ser una herramienta preventiva: saber de antemano si el viaje que vas a hacer pasa por zonas con poca cobertura sanitaria.

2. Objetivo de la Aplicación

El objetivo principal del proyecto es construir una plataforma web que combine mapas interactivos con inteligencia geográfica para responder a una pregunta sencilla pero crítica:



La pregunta clave:

¿Es este viaje seguro desde el punto de vista sanitario?

Para lograrlo, la aplicación tiene que ser capaz de hacer varias cosas:

- Calcular la ruta más rápida entre dos puntos que el usuario elija.
- Analizar esa ruta tramo a tramo y determinar qué tan cerca está cada tramo de un hospital.
- Mostrar esa información de forma visual y clara usando un sistema de colores (el semáforo sanitario).
- Avisarte cuando algún tramo sea zona crítica y, si es posible, sugerirte una ruta alternativa o una parada técnica en un punto seguro.
- Permitir a los usuarios guardar sus rutas habituales para no tener que calcularlas cada vez.

Objetivo secundario: que todo esto funcione sin depender de servicios de pago. Vamos a usar tecnologías open source (gratuitas y de código abierto) para que el proyecto sea sostenible y replicable en otras regiones.

3. Cómo Funciona el Sistema

El sistema está organizado en tres partes (capas) que se comunican entre sí:

3.1 Lo que el usuario ve: el Frontend (Angular)

El frontend es la interfaz web con la que interactúa el usuario. Está construido con Angular, un framework de JavaScript (en realidad TypeScript) que permite crear aplicaciones web dinámicas y bien organizadas. La pantalla principal muestra un mapa interactivo con:

- Un formulario para introducir el punto de origen y el destino del viaje.
- Marcadores en el mapa con la ubicación de todos los hospitales de la base de datos.
- La ruta calculada dibujada con colores según el nivel de riesgo.

¿Cómo se pinta el mapa?

Usamos Leaflet.js, una librería gratuita para mapas web. Es como Google Maps pero sin pagar. Las imágenes del mapa vienen de OpenStreetMap (el equivalente open source de Google Maps).

3.2 El cerebro: el Backend (Spring Boot)

El backend es el servidor que recibe las peticiones del frontend, procesa la información y devuelve los resultados. Está construido con Spring Boot, un framework de Java muy usado en el mundo profesional.

La lógica más importante del proyecto vive aquí: cuando el frontend le envía los puntos de la ruta, el backend los analiza uno por uno preguntando a la base de datos cuál es el hospital más cercano a cada punto y cuánto tiempo hay hasta él por carretera.

3.3 Donde viven los datos: PostgreSQL + PostGIS

La base de datos está en PostgreSQL, un motor de base de datos relacional gratuito y muy potente. Pero la clave es PostGIS, que es una extensión que le añade capacidades geográficas.

Sin PostGIS tendríamos que calcular las distancias a mano en Java, lo que sería complicado y lento. Con PostGIS, una sola línea de SQL es capaz de encontrar el hospital más cercano a unas coordenadas y devolver la distancia en metros. Eso es lo que hace posible que el cálculo sea tan rápido.

Ejemplo de consulta PostGIS:

```
SELECT nombre, ST_Distance(ubicacion::geography, ST_SetSRID(ST_MakePoint(-6.2, 38.9), 4326)::geography) AS metros FROM hospital ORDER BY metros ASC LIMIT 1; – Esta línea de SQL devuelve el hospital más cercano a las coordenadas dadas.
```

3.4 Los servicios externos gratuitos

Para que el mapa funcione sin pagar cuotas a Google, usamos tres servicios open source:

- OpenStreetMap: proporciona las imágenes del mapa (los tiles). Es la Wikipedia de los mapas, mantenida por voluntarios de todo el mundo.
- OSRM (Open Source Routing Machine): calcula la ruta más rápida entre dos puntos usando datos reales de carretera. Le pasamos las coordenadas de origen y destino y nos devuelve la ruta como una lista de puntos GPS.
- Nominatim: convierte texto en coordenadas. Cuando el usuario escribe "Badajoz" en el buscador, Nominatim nos dice que Badajoz está en latitud 38.88 y longitud -6.97. Esto se llama geocodificación.

4. Usuarios y Roles

La aplicación distingue tres tipos de usuario, cada uno con permisos distintos:

Rol	Puede hacer...	NO puede hacer...
 Usuario Invitado	Usar el mapa, calcular rutas, ver el semáforo de cualquier trayecto y ver información de los hospitales.	Guardar rutas ni tener perfil. Se le pedirá que se registre si quiere más funciones.
 Usuario Registrado	Todo lo del invitado, más: guardar rutas favoritas, configurar un perfil de salud opcional y recibir alertas personalizadas.	Acceder al panel de administración ni modificar datos de hospitales.
 Administrador	Acceso total: actualizar la base de datos de hospitales, gestionar usuarios y ver estadísticas de uso de la aplicación.	<i>(Acceso total, pero con gran responsabilidad sobre la veracidad de los datos geográficos.)</i>

5. Alcance del Proyecto

5.1 Lo que SÍ hacemos (funcionalidades incluidas)

- Calcular rutas entre dos puntos y mostrarlas en el mapa.
- Visualizar los hospitales como marcadores clicables con su información.
- Analizar la ruta tramo a tramo y colorear según el riesgo sanitario.
- Sistema de login y registro con JWT (sesiones seguras).
- Guardar rutas en el perfil del usuario registrado.
- Panel de administración para gestionar hospitales y usuarios.
- Filtrar hospitales por especialidad médica.
- Interfaz responsive (funciona en móvil, tablet y escritorio).

5.2 Lo que NO hacemos (fuera de alcance)

Estas cosas quedan fuera de la versión 1.0. No porque no sean buenas ideas, sino porque hay que ser realistas con el tiempo y los recursos disponibles:

- Navegación GPS giro a giro en tiempo real (eso es un Google Maps completo, no un proyecto de fin de ciclo).
- Información de tráfico en tiempo real (necesitaría APIs de pago o datos continuos).
- Solicitar citas médicas o conectar con el sistema sanitario (integración mucho más compleja).
- App móvil nativa para iOS o Android (la web responsive ya cubre este caso de uso principal).

Importante:

Definir bien qué no está incluido es tan importante como definir lo que sí está. Sirve para evitar malentendidos y para que el equipo sepa exactamente qué tiene que entregar.

6. Requisitos Funcionales (RF)

Un requisito funcional describe una acción concreta que el sistema tiene que ser capaz de realizar. Responde a la pregunta: ¿qué tiene que hacer la aplicación?

ID	¿Qué tiene que hacer?	¿Cómo se implementa?	Prioridad
RF-01	Investigar si existen aplicaciones similares en el mercado y detectar sus puntos débiles.	Búsqueda manual en tiendas de apps y webs. Redactar informe de comparativa.	Alta
RF-02	El usuario puede introducir un origen y un destino y ver la ruta en el mapa.	Formulario Angular + geocodificación Nominatim + conexión OSRM para la polyline.	Alta
RF-03	El mapa muestra todos los hospitales de la base de datos como marcadores.	Consumir el endpoint GET /api/hospitales desde Angular y pintar marcadores con Leaflet.	Alta
RF-04	Para cada tramo de la ruta, el sistema calcula el tiempo al hospital más cercano.	Endpoint Spring Boot que ejecuta consulta PostGIS por cada punto de la ruta.	Crítica
RF-05	Los tramos de la ruta se colorean: verde (<30 min), naranja (30-60 min), rojo (>60 min).	El backend devuelve un array de tramos con su nivel. Angular pinta polilíneas de color.	Alta
RF-06	El usuario registrado puede guardar rutas favoritas con un nombre personalizado.	Endpoint POST /api/rutas/guardar. Tabla RutaGuardada en PostgreSQL.	Media
RF-07	Al hacer clic en un hospital, aparece un popup con su información (nombre, teléfono, especialidades).	Popup de Leaflet con datos del hospital obtenidos de la API.	Media
RF-08	El usuario puede filtrar los hospitales que se muestran en el mapa por especialidad.	Componente de filtro Angular + parámetro de query en el endpoint de hospitales.	Baja
RF-09	El administrador puede añadir, editar y eliminar hospitales desde un panel.	CRUD completo de hospitales protegido con rol ADMIN en Spring Boot.	Media
RF-10	Cuando hay tramos rojos, el sistema propone una ruta alternativa o una parada técnica.	Lógica de alternativas en Spring Boot. Si existe, se muestra al usuario como sugerencia.	Alta

7. Requisitos No Funcionales (RNF)

Los requisitos no funcionales describen CÓMO debe ser el sistema, no qué tiene que hacer. Son igual de importantes que los funcionales: una app que hace todo pero tarda 10 segundos por petición no sirve de nada.

ID	Categoría	¿Qué significa en la práctica?
RNF-01	Usabilidad	Un usuario sin formación técnica tiene que poder calcular su primera ruta en menos de 3 clics. La interfaz debe ser intuitiva, sin manual de instrucciones.
RNF-02	Rendimiento	El cálculo completo (ruta + validación hospitalaria + semáforo) debe tardar menos de 3 segundos en condiciones normales de red. Si pasa de eso, el usuario se impacienta.
RNF-03	Open Source / Coste cero	Toda la aplicación debe funcionar con herramientas gratuitas (OSM, OSRM, PostGIS, Spring Boot, Angular). No puede haber dependencias de APIs de pago.
RNF-04	Precisión geográfica	Los tiempos de viaje tienen que ser reales, calculados por carretera (con OSRM), no en línea recta. Un hospital a 10 km puede estar a 45 minutos si hay una sierra en medio.
RNF-05	Seguridad	Las contraseñas se guardan cifradas con BCrypt (nunca en texto plano). Las sesiones de usuario se gestionan con tokens JWT con fecha de expiración.
RNF-06	Responsivo	La interfaz debe funcionar correctamente en pantallas de móvil (320px) y escritorio (1920px). Se usa Angular Material para facilitar el diseño responsive.
RNF-07	Mantenibilidad	Toda la API REST debe estar documentada con Swagger/OpenAPI y accesible en /swagger-ui.html. Así cualquier desarrollador nuevo sabe qué endpoints existen sin leer el código.

8. Flujo de la Aplicación Paso a Paso

Aquí explicamos qué pasa exactamente desde que el usuario abre la app hasta que ve la ruta coloreada en el mapa. Es importante entender bien este flujo porque es el corazón del proyecto:

1. El usuario accede a la web y escribe su origen y destino en el formulario de búsqueda.
2. Angular llama a Nominatim para convertir los nombres de los lugares en coordenadas GPS (latitud y longitud).
3. Con las coordenadas, Angular llama a OSRM para obtener la ruta más rápida por carretera. OSRM devuelve una polyline: básicamente una lista de cientos de puntos GPS que forman el trazado de la carretera.
4. Angular envía esa lista de puntos al backend de Spring Boot mediante una petición HTTP POST a `/api/validar-ruta`.
5. Spring Boot recibe la lista, la divide en segmentos y para cada punto de la ruta ejecuta una consulta en PostgreSQL con PostGIS: ¿cuánto tiempo hay al hospital más cercano desde aquí?
6. PostGIS devuelve el hospital más cercano y la distancia para cada punto. Spring Boot agrupa los puntos en tramos según el nivel de riesgo (verde, naranja o rojo).
7. El backend devuelve al frontend un JSON con los tramos de la ruta y su color correspondiente.
8. Angular usa Leaflet para pintar cada tramo de la ruta con su color en el mapa.
9. Si hay tramos rojos, el sistema también muestra un mensaje de advertencia y, si existe, una ruta alternativa más segura.

9. El Modelo de Datos

Vamos a guardar tres tipos de información en la base de datos. Aquí tienes las tres tablas principales con sus campos:

9.1 Tabla: usuario

Guarda la información de cada persona que se registra en la app.

Campo	Tipo	Restricción	Para qué sirve
id	BIGINT	PK, AUTO	Identificador único de cada usuario
email	VARCHAR(255)	UNIQUE, NOT NULL	Sirve como nombre de usuario para hacer login
password_hash	VARCHAR(255)	NOT NULL	Contraseña cifrada con BCrypt. Nunca se guarda en claro
nombre	VARCHAR(100)		Nombre visible en la interfaz
rol	ENUM	DEFAULT 'USER'	Puede ser USER o ADMIN
fecha_registro	TIMESTAMP	DEFAULT NOW()	Se rellena automáticamente al crear la cuenta

9.2 Tabla: hospital

Los datos de cada centro hospitalario. El campo más importante es ubicacion, que usa el tipo GEOGRAPHY de PostGIS.

Campo	Tipo	Restricción	Para qué sirve
id	BIGINT	PK	Identificador único
nombre	VARCHAR(255)	NOT NULL	Nombre del centro hospitalario
ubicacion	GEOGRAPHY(Point, 4326)	NOT NULL	Las coordenadas GPS en formato PostGIS
direccion	VARCHAR(500)		Dirección postal completa
telefono_urgencias	VARCHAR(20)		El número de urgencias del centro
tipo	VARCHAR(50)		Si es público o privado
servicios_disponibles	JSONB		Lista de especialidades (pediatría, trauma, etc.)

9.3 Tabla: ruta_guardada

Las rutas que los usuarios registrados han guardado en su perfil. Cada ruta pertenece a un usuario (relación 1:N).

Campo	Tipo	Restricción	Para qué sirve
id	BIGINT	PK, AUTO	Identificador único
id_usuario	BIGINT	FK → usuario	A qué usuario pertenece esta ruta
nombre_ruta	VARCHAR(100)		El nombre que el usuario le da (ej: 'Trabajo')
coord_origen	GEOGRAPHY	NOT NULL	Coordenadas del punto de partida
coord_destino	GEOGRAPHY	NOT NULL	Coordenadas del destino
es_segura	BOOLEAN	DEFAULT false	True si toda la ruta es verde o naranja
fecha_creacion	TIMESTAMP	DEFAULT NOW()	Se rellena automáticamente

10. Arquitectura del Sistema

La aplicación sigue una arquitectura en 3 capas desacopladas. "Desacopladas" significa que cada capa funciona de forma independiente y se comunica con las demás a través de interfaces bien definidas (en este caso, la API REST).

10.1 Capa de Presentación: Frontend (Angular)

Es lo que el usuario ve. Toda la interfaz gráfica: el mapa, los formularios, los botones, los popups... Está construida como una Single Page Application (SPA), es decir, la página no se recarga al cambiar de vista, lo que da una experiencia más fluida.

- Framework: Angular 17+ con TypeScript.
- Mapa: Leaflet.js con el paquete ngx-leaflet para integración con Angular.
- Componentes UI: Angular Material (formularios, sidebars, diálogos...).
- Comunicación asíncrona: RxJS con Observables para manejar las llamadas HTTP sin bloquear la interfaz.

10.2 Capa de Negocio: Backend (Spring Boot)

Es el cerebro. Recibe las peticiones del frontend, aplica las reglas del negocio (como el algoritmo de la Hora de Oro) y habla con la base de datos. Expone una API REST que el frontend consume.

- Framework: Spring Boot (Java 17+).
- Persistencia: Spring Data JPA para mapear las tablas de PostgreSQL a objetos Java sin escribir SQL manual.
- Seguridad: Spring Security + JWT para autenticar a los usuarios y proteger los endpoints.
- Documentación: Swagger/OpenAPI, accesible en /swagger-ui.html una vez arrancado el servidor.



¿Cómo funciona JWT?

Cuando el usuario hace login, el servidor genera un token JWT (un texto largo firmado digitalmente). A partir de ahí, Angular incluye ese token en cada petición HTTP. El servidor verifica la firma para saber que el token no ha sido manipulado, sin necesidad de consultar la base de datos en cada petición.

10.3 Capa de Datos: PostgreSQL + PostGIS

Donde viven todos los datos. PostgreSQL es el motor relacional y PostGIS es su extensión geográfica imprescindible para este proyecto.

- PostgreSQL 15+ como motor relacional.
- PostGIS para los tipos de dato geográficos (GEOGRAPHY) y las funciones de cálculo espacial.
- Recomendamos usar Docker para levantar la base de datos en local durante el desarrollo:
`docker run -e POSTGRES_PASSWORD=dam2024 -p 5432:5432 -d postgis/postgis`

10.4 Tabla de servicios externos

Servicio	Proveedor	URL de uso
Mapa (tiles)	OpenStreetMap	https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
Cálculo de rutas	OSRM Project	http://router.project-osrm.org/route/v1/driving/
Geocodificación	Nominatim (OSM)	https://nominatim.openstreetmap.org/search?format=json

11. Planificación por Sprints

El desarrollo se organiza en 3 sprints de 3 semanas. Seguimos una metodología ágil simplificada: al final de cada sprint debe haber algo funcional que enseñar.

Sprint 1 – Semanas 1 a 3: Los Cimientos

Objetivo: tener el entorno montado, la base de datos con datos reales y el mapa mostrando hospitales.

- Semana 1: Análisis de mercado (RF-01). Setup completo: Angular, Spring Boot, Docker + PostGIS. Creación del repositorio Git.
- Semana 2: Crear y poblar la tabla hospital con datos reales de Extremadura. Endpoint GET /api/hospitales funcionando y probado con Postman.
- Semana 3: Integrar Leaflet en Angular, consumir la API y pintar los marcadores de hospitales en el mapa con sus popups (RF-03, RF-07).

Entregable del sprint: mapa con todos los hospitales pintados y clicables.

Sprint 2 – Semanas 4 a 6: El Motor Geográfico

Objetivo: implementar el cálculo de rutas y el semáforo. Esto es el núcleo del proyecto.

- Semana 4: Formulario de búsqueda en Angular. Geocodificación con Nominatim. Conexión con OSRM para trazar la ruta (RF-02).
- Semana 5: El algoritmo de la Hora de Oro en Spring Boot. Endpoint POST /api/validar-ruta que ejecuta las consultas PostGIS (RF-04).
- Semana 6: Enviar los tramos coloreados al frontend y pintarlos con Leaflet (RF-05). Filtros por especialidad (RF-08). Sugerencia de alternativas (RF-10).

Entregable del sprint: ruta completa con semaforización verde/naranja/roja funcionando.

Sprint 3 – Semanas 7 a 9: Pulido y Seguridad

Objetivo: convertir la herramienta en una aplicación completa, segura y profesional.

- Semana 7: Autenticación con Spring Security + JWT. Pantallas de Login y Registro en Angular.
- Semana 8: Endpoint y funcionalidad de rutas guardadas para usuarios logueados (RF-06). Panel de admin (RF-09).
- Semana 9: Pruebas de rendimiento y responsive. Documentación Swagger definitiva. Preparación de la demo.

Entregable del sprint: aplicación completa, segura, documentada y lista para presentar.

12. Ampliaciones Futuras (v2.0)

Estas ideas quedan fuera de la versión 1.0, pero están documentadas para posibles iteraciones futuras del proyecto:

- Botón SOS: un botón de pánico que envíe la ubicación GPS actual del usuario al hospital más cercano junto con su ficha de salud configurada.
- Tiempos de espera en urgencias: si el sistema sanitario publicara datos abiertos en tiempo real, se podría mostrar cuánto tiempo hay de espera en cada urgencia.
- Modo sin conexión: descarga previa de mapas y datos de hospitales de la región para usarlos cuando no hay cobertura de datos.
- Integración con wearables: conectar con dispositivos como Apple Watch o Garmin para detectar caídas automáticamente y activar la alerta sin intervención del usuario.
- App móvil nativa: versión Android/iOS con React Native o Flutter para aprovechar el GPS nativo y las notificaciones push.
- Expansión a Portugal: Extremadura es frontera con Portugal. Con datos del SNS portugués, se podría cubrir rutas transfronterizas.

13. Consejos para el Equipo de Desarrollo

Algunas lecciones que nos ahorrarán tiempo y quebraderos de cabeza durante el desarrollo:

Primero el backend, luego el frontend

Para cada funcionalidad nueva, asegúrate de que el endpoint de Spring Boot funciona correctamente (pruébalo con Postman) antes de tocar Angular. Si el backend falla, el frontend no puede hacer nada y perderás el doble de tiempo depurando.

Happy Path primero

Que el caso principal (calcular una ruta normal entre dos ciudades) funcione perfectamente antes de gestionar casos especiales (zonas sin carretera, errores de red, etc.). No perfecciones lo excepcional mientras lo normal aún tiene fallos.

PostGIS hace el trabajo pesado

No intentes calcular distancias geográficas en Java. Deja que PostGIS lo haga. Una función `ST_Distance()` en una consulta SQL es más rápida y mantenible que 50 líneas de código Java calculando coordenadas con trigonometría.

No olvides el CORS

Angular corre por defecto en el puerto 4200 y Spring Boot en el 8080. Sin configurar el CORS en el backend (`@CrossOrigin` en los controladores), el navegador bloqueará todas las peticiones del frontend al backend. Es un error muy común y muy fácil de corregir.

♥ Una nota final:

Estáis construyendo algo con un propósito real. La precisión de los datos geográficos que introduzcáis en la base de datos importa de verdad. Un hospital mal ubicado puede hacer que el sistema dé una falsa sensación de seguridad a alguien. Dedicadle tiempo a verificar los datos. Merece la pena.